M.Tech. CSE I Year I Sem – Advanced Data Structues (PC-II)

Unit-I

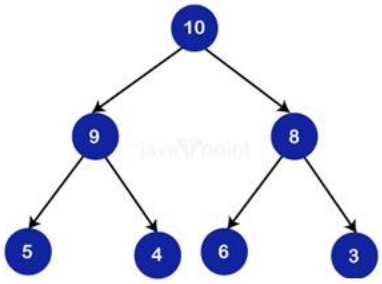
Heap Structures
Introduction, Min-Max Heaps, Leftist trees, Binominal Heaps, Fibonacci heaps

---

LEFTIST TREE / LEFTIST HEAP
Introduction
A priority queue is a leftist heap or leftist tree. It is built using a binary heap variation. For each node, we store the distance to the closest leaf in the subtree anchored at that node. Let us call this value s-value. Compared to a binary heap, which is always a complete binary tree, a leftist heap strives to be extremely imbalanced.

A heap is a tree-based data structure in which the element with the greatest or lowest priority is always kept at the 'root' of the tree.

A heap is a complete binary tree in which all levels except the last must be entirely filled. It is also left-justified, which implies that the left subtree has been filled before the right subtree.



In order to ensure logarithmic performance, the entire binary tree is constantly balanced.

The most widespread misperception regarding heaps is that they are sorted. However, they are only 'partially arranged' and not sorted. Furthermore, the child nodes at each level are unrelated to one another.

The length of the longest subsequence in which the prefix sum at each element stays greater than zero

Introduction

The goal of this issue is to discover the longest subsequence in which the prefix sum at each element remains greater than zero. The method employs MinHeap, a widely used and useful data structure. This blog will assist you in solidifying your understanding of the Min Heap data structure.

Problem Statement

Determine the length of the shortest subsequence such that the prefix sum at each element remains greater than zero given an array of elements.

Case Studies

To better grasp the situation, consider two test cases.

Test case 1:

The output is 4 four the given array





Test case 2:



The output is 3 for the given array



NOTE: In this problem, we must determine the length of the longest subsequence and the longest subarray. A subsequence is a non-contiguous sequence of a given sequence generated by eliminating one or more components from the original sequence.

Approach
The Min Heap data structure will be used to tackle this problem. We will go through the array and place each entry in the minimum heap. We will also add an element to the sum variable, which will keep track of the current element's prefix sum.

If the prefix sum turns negative, we subtract the most negative element from the heap and pop the element in question from the heap.

The Algorithm's Steps

1. Take the user's vector input.
2. Using the priority queue, make an empty Min Heap.
3. Set the sum variable to 0.
4. Run a for loop from 0 to n (the size of the vector).
5. To the sum, add the value of the current vector element.
6. Add the current vector element to the minimum
7. Check to see if the total has become negative.

If the total is negative,
  Subtract the most negative element from the total.
    Remove the top element from the Min Heap.
Else

```
#include<iostream>
using namespace std;
int solve(vector vec, int n)
{
   priority_queue, greater> min_heap; //Creating the Min Heap
   int sum = 0;                                    //Initializing sum as 0
   for (int i = 0; i < n; i++)
   {
      sum += vec[i];        //Adding current element's value to sum
      min_heap.push(vec[i]); //Pushing current element to heap
      if (sum < 0)          //If sum becomes negative
      {
         int a = min_heap.top();
         sum -= a;      //Subtract top element's value from sum
         min_heap.pop(); //Pop the top element from heap
      }
   }
   return min_heap.size(); //return the ans
}
int main()
{
   vector vec;
   int n, x;
   cout << "Enter the size of the array:" << endl;
   cin >> n;
   cout << "Enter the elements:" << endl;
   for (int i = 0; i < n; i++)
   {
      cin >> x;
      vec.push_back(x);
   }
   cout << "Length of the longest subsequence is: " << solve(vec, n);
   return 0;
}
```

Input:
Enter the size of the array:
6
Enter the elements:
-1 6 7 -15 -3 -1
Output

Length of the longest subsequence is: 4

Complexity Analysis
Time Complexity

O(n*logn) since we are iterating through a for loop 'n' times. The priority queue push and pop operations take O(logn) time, making the general time complexity O(n*logn). The number n refers to the total number of items in the input vector.

Space Complexity

O(n), because we're utilizing a priority queue of size n. The number n indicates the number of items in the vector of inputs.

Dry running of test cases

To further understand the problem, consider the dry run of Test Case 1.

The input array is,

-1, 6, 7, -15, -3, -1

At first, sum = 0, and the Min Heap is likewise empty.

-1 is added to the Min Heap, and sum = -1.

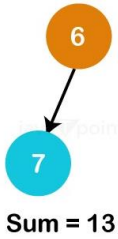


However, as the sum goes negative, we remove the top element through the heap.

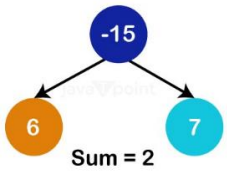
The next piece, 6, is added to a heap, so the resulting sum is 6.



The vector's next member, 7, is then examined. The current sum is 13 (6+7). As a result, 7 is likewise inserted into the heap.



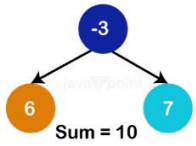
The next ingredient is -15, and adding that renders the total negative.



The total of the elements becomes -2, so we pop the top component of the heap once more, -15.

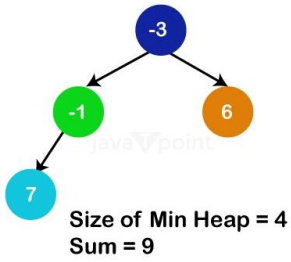
We additionally subtract -15 from our total, making the total 13.

The next member in the vector is -3; adding it to the final sum gives us 10, which is a positive number. As a result, it is added to the heap as well.



We ultimately arrive at the last element in the vector, -1, and add it to the present sum. It is also thrown into the heap.

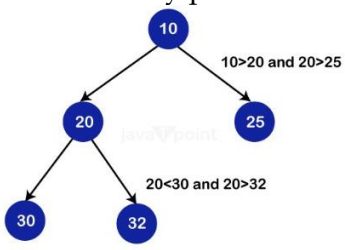
So the present sum is 9, and our Min Heap will appear like this:



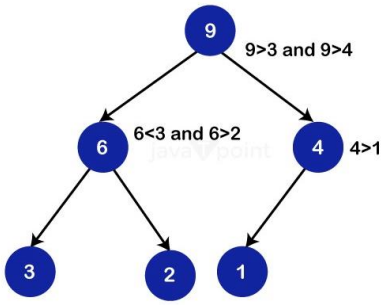
Finally, the size of our Min Heap is 4, which represents the length of the largest subsequence.

Heap Types
There are primarily two sorts of heaps:

Min Heap
In a min heap, the root node represents the smallest of all the nodes in the tree because the value of every parent node is always less than or equal to the number of child nodes.



Max Heap
In a max heap, the value of all the root nodes is the greatest of all the nodes in the tree, and the value of every parent node is always larger than or equal to the value of the child nodes.



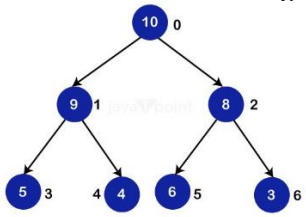
The representation of binary heaps

A binary heap takes up little space and is simple to express in an array. When modelling a binary heap in an algorithm, we should keep the following points in mind:

In the array:

ARR[0]	Root Node
ARR[(i-1)/2]	Parent Node
ARR[2*i+1]	Left Child Node
ARR[2"i+2]	Right Child Node

Consider the following example:



Because the binary tree shown above is complete, we can express it in an array as follows:



Thus, we begin with level 0, i.e., element "10," and fill it up with the first element of the array before incrementing the pointer. We are now at level 1, and we are going to fill the array with the first child node at this level, i.e., the number "9," before moving on to the number "8."

Similarly, we will begin with the first member of level 2 and fill our array appropriately.

A binary heap follows level sequence traversal, which means that each node on a level is visited before proceeding to the next level. In this method, a binary heap can be easily stored in an array.

Binomial Heap Implementation
Operations

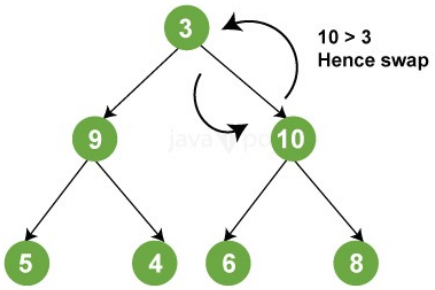
A binary heap can be used for a variety of operations, some of which are described below: (Please keep in mind that the operations outlined below are in Max Heap terminology.)

1. maxHeapify()

MaxHeapify is the programme that's in charge of restoring Max Heap's property. It arranges the node i and its subtrees in such a way that the heap property is preserved.

The function is explained in depth in the following steps:

- Assume we've been given an array called 'ARR' that represents the entire binary tree.
- We make the current element's index, 'i', the 'LARGEST'.
- If ARR[2 * i + 1] > ARR[i], indicating that the child on the left is larger than the present value, the value is set to 'LARGEST'.
- Similarly, if ARR[2 * i + 2] > ARR[i], indicating that the right child is larger than the present value, it is marked as 'LARGEST'.
- Replace the 'LARGEST' element with the current element.
- Rep steps 1-4 until the heap's property is restored.
- Let us practice the preceding stages using an example:



2. insertKey():

During insertion, the new element is attached to the end of the heap and becomes the array's final element. If the recently added element is smaller than the parent, nothing needs to be done. Otherwise, values are swapped until the heap attribute is restored.

3. increaseKey():

It increases the significance of a key at a specific index to some value. If this new value is less than the parent node, anything needs to be done. Otherwise, the heap property is violated, and swaps are performed to restore it.

4. getMax():

Returns the maximum element held at the bottom node of the Max heap.

5. deletekey():

The element to be eliminated is first substituted with positive infinity by using increaseKey(), and then we call removeMax() to remove it from the heap.

The code below depicts the above operations.

```
/ C++ program to depict the implementation of a heap.
#include
using namespace std;
// A class for Max Heap.
class MaxHeap
{
   // A pointer pointing to the elements in the array in the heap.
   int *arr;
   // Maximum possible size of the Max Heap.
   int maxSize;
   // Number of elements in the Max heap currently.
   int heapSize;
public:
   // Constructor function.
   MaxHeap(int maxSize);
   // Heapifies a sub-tree taking the given index as the root.
   void MaxHeapify(int);
   // Returns the index of the parent of the element at ith index.
   int parent(int i)
   {
      return (i - 1) / 2;
   }
   //Returns the index of the left child.
   int lChild(int i)
   {
      return (2 * i + 1);
   }
   // Returns the index of the right child.
   int rChild(int i)
   {
      return (2 * i + 2);
   }
   // Removes the root which in this case contains the maximum element.
   int removeMax();
   // Increases the value of the key given by index i to some new value.
   void increaseKey(int i, int newVal);
   // Returns the maximum key (key at root) from max heap.
   int getMax()
   {
      return arr[0];
   }
   int curSize()
   {
      return heapSize;
```

```
    }
    // Deletes a key at given index i.
    void deleteKey(int i);
    // Inserts a new key 'x' in the Max Heap.
    void insertKey(int x);
};
// Constructor function builds a heap from a given array a[] of the specified size.
MaxHeap::MaxHeap(int totSize)
{
    heapSize = 0;
    maxSize = totSize;
    arr = new int[totSize];
}
// Inserting a new key 'x'.
void MaxHeap::insertKey(int x)
{
    // To check whether the key can be inserted or not.
    if (heapSize == maxSize)
    {
        cout << "\nOverflow: Could not insertKey\n";
        return;
    }
    // The new key is initially inserted at the end.
    heapSize++;
    int i = heapSize - 1;
    arr[i] = x;
    // The max heap property is checked and if violation occurs, it is restored.
    while (i != 0 && arr[parent(i)] < arr[i])
    {
        swap(arr[i], arr[parent(i)]);
        i = parent(i);
    }
}
// Increases value of key at index 'i' to new_val.
void MaxHeap::increaseKey(int i, int newVal)
{
    arr[i] = newVal;
    while (i != 0 && arr[parent(i)] < arr[i])
    {
        swap(arr[i], arr[parent(i)]);
        i = parent(i);
    }
}
// To remove the root node which contains the maximum element of the Max Heap.
int MaxHeap::removeMax()
{
    // Checking whether the heap array is empty or not.
    if (heapSize <= 0)
        return INT_MIN;
```

```
    if (heapSize == 1)
    {
        heapSize--;
        return arr[0];
    }
    // Storing the maximum element to remove it.
    int root = arr[0];
    arr[0] = arr[heapSize - 1];
    heapSize--;
    // To restore the property of the Max heap.
    MaxHeapify(0);
    return root;
}
// In order to delete a key at a given index i.
void MaxHeap::deleteKey(int i)
{
    // It increases the value of the key to infinity and then removes the maximum value.
    increaseKey(i, INT_MAX);
    removeMax();
}
// To heapify the subtree this method is called recursively
void MaxHeap::MaxHeapify(int i)
{
    int l = lChild(i);
    int r = rChild(i);
    int largest = i;
    if (l < heapSize && arr[l] > arr[i])
        largest = l;
    if (r < heapSize && arr[r] > arr[largest])
        largest = r;
    if (largest != i)
    {
        swap(arr[i], arr[largest]);
        MaxHeapify(largest);
    }
}
// Driver program to test above functions.
int main()
{
    // Assuming the maximum size of the heap to be 15.
    MaxHeap h(15);
    // Asking the user to input the keys:
    int k, i, n = 7, arr[10];
    cout << "Enter 7 keys of your choice: \n";
    for (i = 0; i < n; i++)
    {
        cin >> arr[i];
        h.insertKey(arr[i]);
    }
```

```
    // Printing the current size of the heap.
    cout << "The current size of the heap is " << h.curSize() << "\n";
    // Printing the root element which is actually the maximum element.
    cout << "The current maximum element is " << h.getMax() << "\n";
    // Deleting key at index 2.
    h.deleteKey(2);
    // Printing the size of the heap after deletion.
    cout << "The current size of the heap is " << h.curSize() << "\n";
    // Inserting 2 new keys into the heap.
    h.insertKey(15);
    h.insertKey(5);
    cout << "The current size of the heap is " << h.curSize() << "\n";
    cout << "The current maximum element is " << h.getMax() << "\n";
    return 0;
}
```

Input:

```
Enter 7 keys of your choice:
3
10
12
8
4
2
14
```

Output:

```
The current size of the heap is 7.
The current maximum element is 14.
The current size of the heap is 6.
The current size of the heap is 8.
The current maximum element is 15.
```

Time Complexity:

The following section discusses the various time complexities of binary heap functions:

Construction Waste:

Building a heap usually takes O(N) time. The convergence of series can be used to demonstrate it.

Insertion:

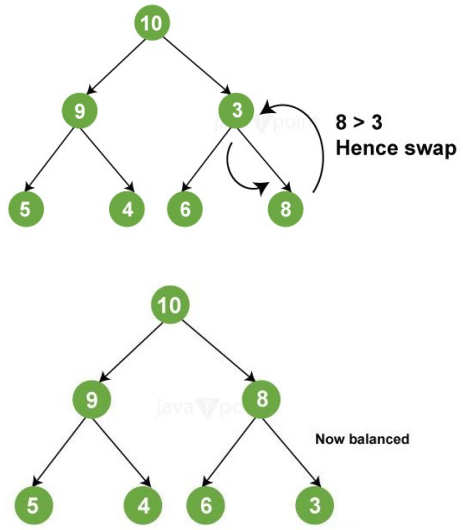
Inserting a new key into the heap necessitates the heapify() operation, requiring O(log (N)) time to restore the tree's order because the height of the tree is O(log (N)). As a result, the time complexity of inserting an additional key is O(log (N)).

Deletion:

After deleting the root node or key value, the hierarchy is heapified to restore the tree's property. As a result, the complexity is similarly given by O (log N).

Obtaining the Minimum or Maximum Element: The root node stores the minimum or maximum element. As a result, it may be retrieved in a single operation, and its time complexity is O(1).

Extracting the Minimum/Maximum element: Because this operation requires calling heapify() after extracting the root element in order to restore the heap's property, the time complexity is O(log(N)).



Applications
Some of the most important binary heap applications are:

- Used in the operating system to prioritise job scheduling.
- Dijkstra's Algorithm (to discover the shortest path), Minimum Spanning Tree, and
- Prim's Algorithm use it.
- It is used to do heap sorting.
- Priority queues are implemented using this method.

Other Applications of Binary Heaps

- Although heaps are relatively adaptable data structures, the allocation of memory is usually very fast. In contrast with other data structures, element addition and removal are also relatively quick.
- The fewest and largest elements can also be found quickly because they are usually stored at the root node.
- Accessing the minimum or maximum element in data structures like nested lists and arrays takes O(N) time. A binary heap, on the other hand, can retrieve the minimum or maximum element in only O(1) time because the maximum or minimum element is always at the root node, and we merely need to return the element at the 0th position of the array.

Time Complexities:

Following are the temporal complexities of several operations on a lefty heap:

Function	Complexity
Get Min	O(1)
Delete Min	O(logN)
Insert Min	O(logN)
Merge	O(logN)

Properties
A lefty heap has the following properties:

Key(i) >= Key(parent(i)): This is comparable to the usual min-heap attribute.
Each node's right descendent has a lower s-value. This signifies that the sum of the number of edges on the shortest route from a node to the final leaf of the right child is less than or equal to that of the left child.

We can reduce the following from the preceding properties:

- The shortest path across any leaf to its base is the one that runs from the root to the highest leaf.
- If there are 'X' nodes in the route from the root to the rightmost leaf, the leftist heap will include at least (2x-1) nodes.
- For a leftist heap with N nodes, the length of the route from the root to the leftmost leaf is also O (logN).

Operations
merge(): The merge operation is the primary operation of a leftist heap.

deleteMin() can be used to replace the minimal node with a merge of the left and right subtrees.

insert(): This is accomplished by executing the merge method on both the initial heap and a new one-node heap.

S-value or Dist

The s-value (or dist) of a node represents the distance that exists between the node in question and the nearest leaf in that node's subtree. A null child's dist is 0 (in certain implementations, the dist for a null child is assumed to be -1).

Merging

Because the right subtree of a leftist heap is smaller than the left subtree, we merge it with the other tree.